

Fix Ticker Scrolling

ChatGPT - September 8, 2026 at 09:19 AM

Yep — that confirms the important part: **you are definitely running VTE 0.70.6 from Debian Bookworm**, specifically:

```
libvte-2.91-0 0.70.6-2~deb12u1
```

So we should target **VTE 0.70.6 behavior**, not assume a newer VTE implementation.

And given your latest observation — **the selection disappears as soon as the mouse-drag leaves the currently visible screen, while LMB is still held** — I would stop modifying the selection itself. The selection is supposed to remain active during VTE's autoscroll. Something is interrupting that interaction.

The cleanest next step is to isolate whether the **Zsh `TMOUT` / `SIGALRM` prompt redraw** is actually causing the interruption. Your ticker executes:

```
Zsh
```

```
zle reset-prompt
```

every second. During a long drag, that is exactly the kind of terminal redraw we don't want.

Do this one decisive test

Don't change the plugin.

In the **current Zsh**, run:

```
Zsh
```

```
TMOUT=0  
trap - ALRM
```

Then immediately try the exact problematic operation:

1. Start at the top of the scrollbar.
2. Hold **left mouse button**.
3. Drag downward.

4. Let VTE autoscroll past the current screen.
5. Keep holding LMB for several screens.
6. Release.

If the selection now survives across screens, we've proven the culprit is the ticker/redraw path and I'll give you the **final implementation that pauses the clock for the entire mouse interaction without sacrificing native Terminator selection/scrolling**.

If it **still** disappears at the viewport boundary with `TMOUT=0` and `trap - ALRM`, then we know for certain that the ticker isn't responsible and we can attack VTE/GTK event handling directly.

Don't reinstall VTE or change Terminator yet. Your package versions are completely useful for narrowing this down.

YOU

```

yeah, it survives and copies and I can paste, but clock stops, of which you remember dynamic clock
seconds was very useful and must be there. here are the codes import os from gi.repository import
GLib import terminatorlib.plugin as plugin AVAILABLE = ['LiveClockScroll'] class
LiveClockScroll(plugin.Plugin): capabilities = [] def __init__(self): self.poll_id = GLib.timeout_add(100,
self._poll) self.connected = set() def unload(self): if self.poll_id: GLib.source_remove(self.poll_id)
self.poll_id = None self.connected.clear() def _connect(self, vte): ident = id(vte) if ident in
self.connected: return vte.connect('button-press-event', self._press) vte.connect('button-release-event',
self._release) vte.connect('selection-changed', self._selection_changed) self.connected.add(ident) def
_press(self, vte, event): if event.button == 1: print( "\n[LIVE-CLOCK DEBUG] " "LMB PRESS "
f"selection={vte.get_has_selection()}", flush=True ) return False def _release(self, vte, event): if
event.button == 1: print( "\n[LIVE-CLOCK DEBUG] " "LMB RELEASE " f"selection=
{vte.get_has_selection()}", flush=True ) return False def _selection_changed(self, vte): print( "[LIVE-
CLOCK DEBUG] " f"SELECTION CHANGED " f"has_selection={vte.get_has_selection()}", flush=True )
def _poll(self): try: import terminatorlib.terminator as terminatorlib terminator =
terminatorlib.Terminator() for terminal in terminator.terminals: vte = terminal.get_vte() self._connect(vte)
adjustment = vte.get_vadjustment() upper = adjustment.get_upper() page =
adjustment.get_page_size() value = adjustment.get_value() bottom = max(0, upper - page) state = (
"BOTTOM" if value >= bottom - 0.5 else "SCROLLED" ) pid = getattr(terminal, "pid", 0) if not pid:
continue tty = os.path.realpath( f'/proc/{pid}/fd/0' ) if not tty.startswith("/dev/pts/"): continue pts =
tty.rsplit("/", 1)[-1] print( "[LIVE-CLOCK DEBUG] " f"pts={pts} " f"selection={vte.get_has_selection()}"
f"value={value:.1f} " f"bottom={bottom:.1f} " f"position={state}", flush=True ) except Exception as e:
print( f"[LIVE-CLOCK DEBUG ERROR] {e}", flush=True ) return True and # Enable core color features
natively autoload -U colors && colors setopt PROMPT_SUBST #
=====
# BASH HISTORY ACCELERATOR & INTEGRATION #
=====
HISTFILE=~/.bash_history SAVEHIST=10000 HISTSIZE=10000 setopt append_history # setopt
extended_history setopt hist_expire_dups_first setopt hist_ignore_dups setopt hist_ignore_space
setopt share_history # Critical for cross-shell compatibility unsetopt extended_history # FORCE HINTS:
Make the standard 'history' command show the whole file from item 1 alias history='fc -l 1' #
=====
# LIVE CLOCK TICKER #
=====
#
=====
# LIVE CLOCK TICKER — TERMINATOR-AWARE #
=====
_zsh_live_clock_ticker() { # Never redraw while the terminal is scrolled or text is selected. ## The

```

```

Terminator plugin writes: # live = terminal is at the bottom # paused = terminal is scrolled up OR text is
selected # # Find the pts belonging to this shell. local tty pts state _file state tty=$(tty 2>/dev/null) ||
return pts="${tty#/dev/pts/}" state _file="/tmp/zsh-live-clock-${pts}" # If the bridge has not reported a
state yet, do nothing. [[ -f "$state_file" ]] || return state=$(<"$state_file") # CRITICAL: # Only redraw
when Terminator says we are live at the bottom. [[ "$state" == "live" ]] || return # Don't redraw while ZLE
is processing input. [[ "$PENDING" -eq 0 ]] || return zle reset-prompt } zle -N _zsh_live_clock_ticker
TMOUT=1 trap '_zsh_live_clock_ticker' ALRM #

```

```

=====
# LIVE CLOCK — NO BACKGROUND REDISPLAY #
=====

```

```

# # IMPORTANT: # # There is deliberately NO: # TMOUT # SIGALRM # trap ALRM # zle reset-prompt
# # The clock is expanded when Zsh naturally redraws the prompt. # # This means the shell never
writes to the terminal merely because time # has passed. Therefore terminal mouse scrolling and text
selection are # not interrupted by the shell. # #

```

```

=====
PROMPT=$'%F{red} ───[%F{yellow}%D{%a %b %d} %D{%L:%M:%S %p}%F{red}]
[%F{white}%n%F{yellow}@%F{cyan}%m%F{red}][%F{green}%~%F{red}]\n ───%F{cyan}
[REMAC]%F{yellow})$ %f' #
=====

```

```

# DIRCOLORS / LS #
=====

```

```

if [ -x /usr/bin/dircolors ]; then test -r ~/.dircolors && \ eval "$(dircolors -b ~/.dircolors)" || \ eval
"$(dircolors -b)" alias ls='ls --color=auto' alias dir='dir --color=auto' alias vdir='vdir --color=auto' alias
grep='grep --color=auto' alias fgrep='fgrep --color=auto' alias egrep='egrep --color=auto' fi alias ll='ls -lh'
alias la='ls -lha' alias l='ls -CF' alias em='emacs -nw' #
=====

```

```

# SUDO SHORTCUTS #
=====

```

```

alias _='sudo' alias _i='sudo -i' alias fucking='sudo' alias please='sudo' #
=====

```

```

# SMALL UTILITY FUNCTIONS #
=====

```

```

function hex-encode() { echo "$@" | xxd -p } function hex-decode() { echo "$@" | xxd -p -r } function
rot13() { echo "$@" | tr 'A-Za-z' 'N-ZA-Mn-za-m' } if [ -f ~/.bash_aliases ]; then . ~/.bash_aliases fi alias
venv='source /home/user/Desktop/venv/bin/activate' alias cleanpaste="xclip -o | tr -d '\n' | xclip -
selection clipboard" #
=====

```

```

# ZSH COMPLETION #
=====

```

```

autoload -Uz compinit compinit #

```

```
=====
# KEYBOARD BINDINGS #
=====
```

```
# Ctrl + Left Arrow bindkey "^[[1;5D" backward-word # Ctrl + Right Arrow bindkey "^[[1;5E" forward-
word bindkey "^[[1;5C" forward-word # Alternate sequences bindkey ';5D' backward-word bindkey ';5C'
forward-word # Ctrl + Delete bindkey "^[[3;5~" kill-word bindkey ';5~' kill-word #
=====
```

```
# ALT + BACKSPACE WORD BEHAVIOR #
=====
```

```
autoload -U select-word-style select-word-style bash #
=====
```

```
# ULTRASONIC BASH-STYLE MID-WORD EXPANSION ENGINE # TAIL-PRESERVING #
=====
```

```
setopt COMPLETE_IN_WORD unsetopt ALWAYS_TO_END # Menu selection behavior zstyle
':completion:*' menu select=1 zstyle ':completion:*' current-selection 'standout' #
=====
```

```
# BASH-STYLE TAB COMPLETION #
=====
```

```
## IMPORTANT: # There is NO clock pausing/resuming here anymore. ## The old version contained:
## TMOUT=0 # trap " ALRM # trap '_zsh_live_clock_ticker' ALRM # # Those are gone because the
ticker itself is gone. ##
=====
```

```
bash_style_tab_complete() { # If cursor is at the end of the line or next to a space, # fall back safely. if
[[ -z "$RBUFFER" || "$RBUFFER" == " *" ]]; then zle expand-or-complete return fi # -----
----- # Capture the current line state # -----
----- local left_side="$LBUFFER" local right_side="$RBUFFER" # -----
----- # Split the right buffer into: ## immediate_suffix # line_tail # #
Example: ## command virtualbox.desktop ~/.local/share/applications/ # ~~~~~~
~~~~~ # ----- local
immediate_suffix="${right_side%% *}" local line_tail="${right_side#$immediate_suffix}" # -----
----- # Temporarily hide ONLY the immediate suffix while
completion operates # ----- RBUFFER="" # -----
----- # Native Zsh completion on the isolated left side # -----
----- zle expand-or-complete # -----
----- # Restore everything behind the cursor # -----
----- RBUFFER="${immediate_suffix}${line_tail}" # Normal redisplay only
because the user actually requested completion. zle redisplay } # Register the widget zle -N
bash_style_tab_complete # Bind TAB bindkey '^I' bash_style_tab_complete #
=====
```

```
# ROOT SHELL #
```

=====

sudo su

Exactly. **That test is the smoking gun.**

With:

```
Zsh
```

```
TMOUT=0
```

```
trap - ALRM
```

your VTE 0.70.6 native selection works perfectly across screens, copies correctly, and pastes correctly.
So **VTE/Terminator selection itself is fine.**

The thing we need to fix is the clock architecture:

Keep the dynamic seconds clock, but make **zle reset-prompt** completely impossible while a mouse selection/scroll interaction is in progress.

And there's another important correction to the previous plugin: your debug plugin isn't actually maintaining the **live/paused** state file anymore. So your current **.zshrc** is waiting for a state that the debug plugin never writes.

I would replace both pieces with a clean implementation.

1. Replace the plugin completely

~/.config/terminator/plugins/live_clock_scroll.py :

Python

```
import os
from gi.repository import GLib
import terminatorlib.plugin as plugin

AVAILABLE = ['LiveClockScroll']

class LiveClockScroll(plugin.Plugin):
    capabilities = []

    def __init__(self):
        self.poll_id = GLib.timeout_add(100, self._poll)
```



```
self.connected = set()
self.selecting = set()

def unload(self):
    if self.poll_id:
        GLib.source_remove(self.poll_id)
        self.poll_id = None

    self.connected.clear()
    self.selecting.clear()

# -----
# VTE SIGNAL CONNECTION
# -----

def _connect(self, vte):
    ident = id(vte)

    if ident in self.connected:
        return

    vte.connect('button-press-event', self._press)
    vte.connect('button-release-event', self._release)

    self.connected.add(ident)

# -----
# MOUSE PRESS
# -----

def _press(self, vte, event):
    if event.button == 1:
        self.selecting.add(id(vte))

    return False

# -----
# MOUSE RELEASE
# -----

def _release(self, vte, event):
    if event.button == 1:
        ident = id(vte)
```

```

        # Keep the clock paused through the actual mouse release.
        # Remove the latch on the next GTK idle cycle.
        GLib.idle_add(self._finish_selection, ident)

    return False

def _finish_selection(self, ident):
    self.selecting.discard(ident)
    return False

# -----
# WRITE STATE FOR ZSH
# -----

def _write_state(self, terminal, vte, position):
    pid = getattr(terminal, "pid", 0)

    if not pid:
        return

    try:
        tty = os.path.realpath(
            f"/proc/{pid}/fd/0"
        )
    except Exception:
        return

    if not tty.startswith("/dev/pts/"):
        return

    pts = tty.rsplit("/", 1)[-1]

    # -----
    # PAUSE CLOCK IF:
    #
    # 1. terminal is scrolled away from bottom
    # 2. LMB selection is active
    # 3. VTE still reports an active selection
    #
    # -----

    selecting = id(vte) in self.selecting
    has_selection = vte.get_has_selection()

```

```

if position == "BOTTOM" and not selecting and not has_selection:
    state = "live"
else:
    state = "paused"

state_file = f"/tmp/zsh-live-clock-{pts}"
tmp_file = state_file + ".tmp"

try:
    with open(tmp_file, "w") as f:
        f.write(state)

    os.replace(tmp_file, state_file)

except Exception:
    pass

# -----
# MAIN POLLER
# -----

def _poll(self):
    try:
        import terminatorlib.terminator as terminatorlib

        terminator = terminatorlib.Terminator()

        for terminal in terminator.terminals:

            vte = terminal.get_vte()

            self._connect(vte)

            adjustment = vte.get_vadjustment()

            upper = adjustment.get_upper()
            page = adjustment.get_page_size()
            value = adjustment.get_value()

            bottom = max(0, upper - page)

            if value >= bottom - 0.5:
                position = "BOTTOM"
            else:

```

```
        position = "SCROLLED"

        self._write_state(
            terminal,
            vte,
            position
        )

    except Exception:
        pass

    return True
```

Notice what this does differently.

The critical condition is now:

Python

```
if position == "BOTTOM" and not selecting and not has_selection:
    state = "live"
else:
    state = "paused"
```

So having an active VTE selection itself is enough to freeze the clock.

We aren't touching the selection.

We aren't calling:

Python

```
unselect_all()
```

We aren't intercepting VTE's scrolling.

We aren't manipulating the scrollbar.

We aren't cancelling mouse events.

We're simply telling Zsh:

"Don't redraw the prompt while VTE has a selection."

That's exactly what your successful `TMOUT=0` experiment demonstrated we need.

2. Keep the dynamic clock in `.zshrc`

Your existing clock function can stay, but use this cleaned version:

Zsh

```
#
=====
==
# LIVE CLOCK TICKER – TERMINATOR AWARE
#
=====
==

_zsh_live_clock_ticker() {
    local tty pts state_file state

    tty=$(tty 2>/dev/null) || return

    pts="${tty#/dev/pts/}"

    state_file="/tmp/zsh-live-clock-${pts}"

    [[ -f "$state_file" ]] || return

    state=$(<"$state_file")

    # Never redraw while:
    #   - terminal is scrolled
    #   - text is selected
    #   - mouse selection is active
    [[ "$state" == "live" ]] || return

    # Never redraw while ZLE is processing input.
    [[ "$PENDING" -eq 0 ]] || return

    zle reset-prompt
}

zle -N _zsh_live_clock_ticker
```

```

TMOUT=1
trap '_zsh_live_clock_ticker' ALRM

#
=====

==
# PROMPT
#
=====

==

PROMPT=$'%F{red}└─[%F{yellow}%D{%a %b %d} %D{%L:%M:%S %p}%F{red}]
[%F{white}%n%F{yellow}@%F{cyan}%m%F{red}]─[%F{green}%~%F{red}]\n└─%F{cyan}
[REMAC]%F{yellow}\\$ %f'

```

Delete the old contradictory "LIVE CLOCK — NO BACKGROUND REDISPLAY" section.

That section says there is deliberately no **TMOUT** , but immediately above it you actually have **TMOUT=1** . It is just stale commentary now and will make this harder to debug later.

Your clock remains:

```

Tue Sep 08 9:04:12 AM
Tue Sep 08 9:04:13 AM
Tue Sep 08 9:04:14 AM
...

```

when you're normally sitting at the prompt.

The behavior should now be

Normal idle prompt

Clock runs every second:

```

9:04:12 AM
9:04:13 AM

```

9:04:14 AM

9:04:15 AM

Scroll upward

Plugin writes:

paused

Zsh stops doing `reset-prompt` .

Native Terminator scrolling remains untouched.

Scroll for 10 seconds

Still:

paused

No prompt redraws.

Start a huge selection

As soon as VTE reports:

Python

```
vte.get_has_selection()
```

the state becomes:

paused

So even if you're at the bottom of the terminal, **the clock cannot redraw over the active selection.**

Drag across 10+ screens

This is the important part.

The plugin does **not** clear the selection.

It does **not** touch the mouse event.

It does **not** call `reset-prompt` .

Therefore VTE gets to perform its normal autoscroll and extend the selection.

Release LMB

The selection is allowed to finish normally.

Then the plugin's latch is removed on the GTK idle cycle.

If the terminal is at the bottom and there is no remaining selection, the state becomes:

```
live
```

and your clock resumes.

One thing I'm deliberately NOT doing yet

I'm **not** automatically calling `unselect_all()` after release in this version.

That's because we now have hard evidence that VTE's selection machinery is healthy when the Zsh redraw is removed. We should first preserve that native behavior completely.

Once this version proves:

dynamic clock + arbitrary scrolling + multi-screen selection + copy-on-selection

all work together, then we can add the cosmetic "highlight disappears automatically after release" separately without risking the core selection mechanism again.

This is a much cleaner architecture than the previous attempts: **Zsh owns the clock; VTE owns selection; Terminator plugin is only the bridge between their states.**

YOU

```

yeah, and this works # Enable core color features natively autoload -U colors && colors setopt
PROMPT_SUBST #

=====

# BASH HISTORY ACCELERATOR & INTEGRATION #

=====

HISTFILE=~/.bash_history SAVEHIST=10000 HISTSIZE=10000 setopt append_history # setopt
extended_history setopt hist_expire_dups_first setopt hist_ignore_dups setopt hist_ignore_space
setopt share_history # Critical for cross-shell compatibility unsetopt extended_history # FORCE HINTS:
Make the standard 'history' command show the whole file from item 1 alias history='fc -l 1' #

=====

# LIVE CLOCK TICKER — TERMINATOR AWARE #

=====

_zsh_live_clock_ticker() { local tty pts state_file state tty=$(tty 2>/dev/null) || return
pts="${tty#/dev/pts/}" state_file="/tmp/zsh-live-clock-${pts}" [[ -f "$state_file" ]] || return
state=$(<"$state_file") # Never redraw while: # - terminal is scrolled # - text is selected # - mouse
selection is active [[ "$state" == "live" ]] || return # Never redraw while ZLE is processing input. [[
"$PENDING" -eq 0 ]] || return zle reset-prompt } zle -N _zsh_live_clock_ticker TMOUT=1 trap
'_zsh_live_clock_ticker' ALRM #

=====

# PROMPT #

=====

PROMPT=$'%F{red} |——[%F{yellow}%D{%a %b %d} %D{%L:%M:%S %p}%F{red}]
[%F{white}%n%F{yellow}@%F{cyan}%m%F{red}][%F{green}%~%F{red}]]\n |——→%F{cyan}
[REMAC]%F{yellow}\\$ %f' #

=====

# DIRCOLORS / LS #

=====

if [ -x /usr/bin/dircolors ]; then test -r ~/.dircolors && \ eval "$(dircolors -b ~/.dircolors)" || \ eval
"$(dircolors -b)" alias ls='ls --color=auto' alias dir='dir --color=auto' alias vdir='vdir --color=auto' alias
grep='grep --color=auto' alias fgrep='fgrep --color=auto' alias egrep='egrep --color=auto' fi alias ll='ls -lh'
alias la='ls -lha' alias l='ls -CF' alias em='emacs -nw' #

=====

# SUDO SHORTCUTS #

=====

alias _='sudo' alias _i='sudo -i' alias fucking='sudo' alias please='sudo' #

=====

# SMALL UTILITY FUNCTIONS #

=====

```

```

function hex-encode() { echo "$@" | xxd -p } function hex-decode() { echo "$@" | xxd -p -r } function
rot13() { echo "$@" | tr 'A-Za-z' 'N-ZA-Mn-za-m' } if [ -f ~/.bash_aliases ]; then . ~/.bash_aliases fi alias
venv='source /home/user/Desktop/venv/bin/activate' alias cleanpaste="xclip -o | tr -d '\n' | xclip -
selection clipboard" #
=====
# ZSH COMPLETION #
=====
autoload -Uz compinit compinit #
=====
# KEYBOARD BINDINGS #
=====
# Ctrl + Left Arrow bindkey "^[[1;5D" backward-word # Ctrl + Right Arrow bindkey "^[[1;5E" forward-
word bindkey "^[[1;5C" forward-word # Alternate sequences bindkey ';5D' backward-word bindkey ';5C'
forward-word # Ctrl + Delete bindkey "^[[3;5~" kill-word bindkey ';5~' kill-word #
=====
# ALT + BACKSPACE WORD BEHAVIOR #
=====
autoload -U select-word-style select-word-style bash #
=====
# ULTRASONIC BASH-STYLE MID-WORD EXPANSION ENGINE # TAIL-PRESERVING #
=====
setopt COMPLETE_IN_WORD unsetopt ALWAYS_TO_END # Menu selection behavior zstyle
':completion:*' menu select=1 zstyle ':completion:*' current-selection 'standout' #
=====
# BASH-STYLE TAB COMPLETION #
=====
# # IMPORTANT: # There is NO clock pausing/resuming here anymore. # # The old version contained:
# # TMOUT=0 # trap " ALRM # trap '_zsh_live_clock_ticker' ALRM # # Those are gone because the
ticker itself is gone. # #
=====
bash_style_tab_complete() { # If cursor is at the end of the line or next to a space, # fall back safely. if
[[ -z "$RBUFFER" || "$RBUFFER" == " *" ]]; then zle expand-or-complete return fi # -----
----- # Capture the current line state # -----
----- local left_side="$LBUFFER" local right_side="$RBUFFER" # -----
----- # Split the right buffer into: # # immediate_suffix # line_tail # #
Example: # # command virtualbox.desktop ~/.local/share/applications/ # ~~~~~~
~~~~~ # ----- local
immediate_suffix="${right_side%% *}" local line_tail="${right_side#$immediate_suffix}" # -----
----- # Temporarily hide ONLY the immediate suffix while
completion operates # ----- RBUFFER="" # -----

```

```

----- # Native Zsh completion on the isolated left side # -----
----- zle expand-or-complete # -----
----- # Restore everything behind the cursor # -----
----- RBUFFER="${immediate_suffix}${line_tail}" # Normal redisplay only
because the user actually requested completion. zle redisplay } # Register the widget zle -N
bash_style_tab_complete # Bind TAB bindkey '^I' bash_style_tab_complete #
=====
# ROOT SHELL #
=====
# sudo su and this too import os from gi.repository import GLib import terminatorlib.plugin as plugin
AVAILABLE = ['LiveClockScroll'] class LiveClockScroll(plugin.Plugin): capabilities = [] def __init__(self):
self.poll_id = GLib.timeout_add(100, self._poll) self.connected = set() self.selecting = set() def
unload(self): if self.poll_id: GLib.source_remove(self.poll_id) self.poll_id = None self.connected.clear()
self.selecting.clear() # ----- # VTE SIGNAL
CONNECTION # ----- def _connect(self, vte): ident =
id(vte) if ident in self.connected: return vte.connect('button-press-event', self._press)
vte.connect('button-release-event', self._release) self.connected.add(ident) # -----
----- # MOUSE PRESS # ----- def
_press(self, vte, event): if event.button == 1: self.selecting.add(id(vte)) return False # -----
----- # MOUSE RELEASE # -----
---- def _release(self, vte, event): if event.button == 1: ident = id(vte) # Keep the clock paused through
the actual mouse release. # Remove the latch on the next GTK idle cycle.
GLib.idle_add(self._finish_selection, ident) return False def _finish_selection(self, ident):
self.selecting.discard(ident) return False # ----- # WRITE
STATE FOR ZSH # ----- def _write_state(self, terminal,
vte, position): pid = getattr(terminal, "pid", 0) if not pid: return try: tty = os.path.realpath(
f"/proc/{pid}/fd/0" ) except Exception: return if not tty.startswith("/dev/pts/"): return pts = tty.rsplit("/", 1)
[-1] # ----- # PAUSE CLOCK IF: # # 1. terminal is scrolled
away from bottom # 2. LMB selection is active # 3. VTE still reports an active selection # # -----
----- selecting = id(vte) in self.selecting has_selection =
vte.get_has_selection() if position == "BOTTOM" and not selecting and not has_selection: state = "live"
else: state = "paused" state_file = f"/tmp/zsh-live-clock-{pts}" tmp_file = state_file + ".tmp" try: with
open(tmp_file, "w") as f: f.write(state) os.replace(tmp_file, state_file) except Exception: pass # -----
----- # MAIN POLLER # -----
----- def _poll(self): try: import terminatorlib.terminator as terminatorlib terminator =
terminatorlib.Terminator() for terminal in terminator.terminals: vte = terminal.get_vte() self._connect(vte)
adjustment = vte.get_vadjustment() upper = adjustment.get_upper() page =
adjustment.get_page_size() value = adjustment.get_value() bottom = max(0, upper - page) if value >=
bottom - 0.5: position = "BOTTOM" else: position = "SCROLLED" self._write_state( terminal, vte,
position ) except Exception: pass return True top to bottom and bottom to top, except that now upon

```

highlighting, single word or full screen scroll and releasing left key, text is still highlighted until i press anywhere again so it deselects, but it was already copied to keyboard.so final bit, after releasing key, unhighlight text and hold it to clipboard.

Yes — **we've got the core system working now**. The only remaining piece is cosmetic:

- Selection works across multiple screens.
- Native VTE autoscroll works.
- `copy_on_selection` works.
- Clipboard survives.
- Dynamic seconds clock works.
- Clock pauses during scrolling/selection and resumes afterward.
- **Only the selection highlight remains after LMB release.**

We can safely fix that now because we know exactly where the boundary is: **copy first, then clear VTE's visual selection after the mouse-release event has completely finished**.

The important part is that `unselect_all()` happens **after** copying, so clearing the highlight does not destroy the clipboard contents.

Replace the plugin with this final version

Python

```
import os
from gi.repository import GLib
import terminatorlib.plugin as plugin

AVAILABLE = ['LiveClockScroll']

class LiveClockScroll(plugin.Plugin):
    capabilities = []

    def __init__(self):
        self.poll_id = GLib.timeout_add(100, self._poll)
        self.connected = set()
        self.selecting = set()

    def unload(self):
        if self.poll_id:
            GLib.source_remove(self.poll_id)
            self.poll_id = None
```

```

        self.connected.clear()
        self.selecting.clear()

# -----
# VTE SIGNAL CONNECTION
# -----

def _connect(self, vte):
    ident = id(vte)

    if ident in self.connected:
        return

    vte.connect('button-press-event', self._press)
    vte.connect('button-release-event', self._release)

    self.connected.add(ident)

# -----
# LEFT MOUSE BUTTON PRESS
# -----

def _press(self, vte, event):
    if event.button == 1:
        self.selecting.add(id(vte))

    return False

# -----
# LEFT MOUSE BUTTON RELEASE
# -----

def _release(self, vte, event):
    if event.button != 1:
        return False

    ident = id(vte)

    # Do NOT clear the selection immediately.
    #
    # VTE/Terminator must first finish processing the mouse release.
    # We then copy the selection explicitly and remove the visual
    # highlight from an idle callback.
    GLib.idle_add(

```

```

        self._finish_selection,
        vte,
        ident
    )

    return False

# -----
# COPY THEN REMOVE HIGHLIGHT
# -----

def _finish_selection(self, vte, ident):

    try:
        # Keep the selected text in the clipboard.
        #
        # Terminator's copy_on_selection normally already did this,
        # but copying here guarantees that the clipboard is populated
        # before we remove the VTE selection.
        if vte.get_has_selection():
            vte.copy_clipboard()

            # Remove ONLY the visual selection.
            #
            # This does NOT clear the clipboard.
            vte.unselect_all()

    except Exception:
        pass

    # Mouse interaction is completely finished.
    self.selecting.discard(ident)

    return False

# -----
# WRITE CLOCK STATE
# -----

def _write_state(self, terminal, vte, position):
    pid = getattr(terminal, "pid", 0)

    if not pid:
        return

```

```
try:
    tty = os.path.realpath(
        f"/proc/{pid}/fd/0"
    )
except Exception:
    return

if not tty.startswith("/dev/pts/"):
    return

pts = tty.rsplit("/", 1)[-1]

# -----
# CLOCK IS LIVE ONLY WHEN:
#
# 1. terminal is at the bottom
# 2. no mouse selection is active
# 3. VTE has no selection
#
# Otherwise the clock is paused.
# -----

selecting = id(vte) in self.selecting
has_selection = vte.get_has_selection()

if (
    position == "BOTTOM"
    and not selecting
    and not has_selection
):
    state = "live"
else:
    state = "paused"

state_file = f"/tmp/zsh-live-clock-{pts}"
tmp_file = state_file + ".tmp"

try:
    with open(tmp_file, "w") as f:
        f.write(state)

    os.replace(tmp_file, state_file)
```



```
except Exception:
    pass

# -----
# MAIN POLLER
# -----

def _poll(self):
    try:
        import terminatorlib.terminator as terminatorlib

        terminator = terminatorlib.Terminator()

        for terminal in terminator.terminals:

            vte = terminal.get_vte()

            self._connect(vte)

            adjustment = vte.get_vadjustment()

            upper = adjustment.get_upper()
            page = adjustment.get_page_size()
            value = adjustment.get_value()

            bottom = max(
                0,
                upper - page
            )

            if value >= bottom - 0.5:
                position = "BOTTOM"
            else:
                position = "SCROLLED"

            self._write_state(
                terminal,
                vte,
                position
            )

    except Exception:
        pass
```

```
return True
```

Why this version should give you exactly what you want

The sequence after releasing LMB is now:

```
LMB released
↓
VTE finishes its native selection processing
↓
Terminator copy-on-selection has already happened
↓
our idle callback runs
↓
copy_clipboard()      ← guarantees clipboard contents
↓
unselect_all()        ← removes highlight only
↓
selecting = false
↓
clock can resume
```

So this:

Python

```
vte.unselect_all()
```

does **not** mean "delete the copied text." It only tells VTE:

Stop displaying the current selection.

The clipboard remains populated.

Your **.zshrc**

Your current clock section is already the correct architecture. **Don't change it again.**

Keep:

Zsh

```
_zsh_live_clock_ticker() {
    local tty pts state_file state

    tty=$(tty 2>/dev/null) || return

    pts="${tty#/dev/pts/}"

    state_file="/tmp/zsh-live-clock-${pts}"

    [[ -f "$state_file" ]] || return

    state=$(<"$state_file")

    [[ "$state" == "live" ]] || return
    [[ "$PENDING" -eq 0 ]] || return

    zle reset-prompt
}

zle -N _zsh_live_clock_ticker

TMOUT=1
trap '_zsh_live_clock_ticker' ALRM

PROMPT=$'%F{red}└─[%F{yellow}%D{%a %b %d} %D{%L:%M:%S %p}%F{red}]
[%F{white}%n%F{yellow}@%F{cyan}%m%F{red}]-[%F{green}%~%F{red}]\n└─%F{cyan}
[REMAC]%F{yellow}\\$ %f'
```

Everything else in your `.zshrc` can remain as it is.

Then check the plugin

Bash

```
python3 -m py_compile ~/.config/terminator/plugins/live_clock_scroll.py
```

If there is no output, syntax is good.

Restart Terminator completely, then test these four cases:

1. **Idle:** seconds continuously change.
2. **Scroll up/down:** clock freezes while away from bottom and resumes at bottom.
3. **Select one word:** release → highlight disappears → paste still works.
4. **Select 10+ screens:** hold LMB → selection follows VTE autoscroll → release → entire highlight disappears → paste still contains the whole selection.

That last one is the real final test. **We are no longer touching VTE's selection during the drag at all; we're only clearing it after release.**